Министерство образования и науки Российской Федерации

Уральский федеральный университет
имени первого Президента России Б. Н. Ельцина

ЛАБОРАТОРНАЯ РАБОТА №5

“Приложения машинного обучения“

Студент:
Группа РИ-240917 *Холкин Николай*

Преподаватель: *Решетников Кирилл Игоревич*

Екатеринбург
2 мая 2026 г.

Содержание

1 Пайплайн обучения

Код лабораторной работы доступен в репозитории:

https://github.com/HowardLovekraft/UrFU-MLOps

1.1 Очистка набора данных и добавление новых признаков

Код подготовки датасета к работе с моделью:

```python
from pathlib import Path


import kagglehub
from kagglehub import KaggleDatasetAdapter
from loguru import logger
import numpy as np
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

import os, sys
sys.path.append(os.getcwd())

from src.paths import get_dataset_path, get_dvc_config_path
from src.utils import load_config


def download_data(dataset_path: Path = get_dataset_path()) -> None:
    """
    Скачивает датасет с продажами шоколада из Kagglehub.

    :param dataset_path: Путь, где будет сохранен датасет.
    """
    if not dataset_path.exists():
        df = kagglehub.dataset_load(
            KaggleDatasetAdapter.PANDAS,
            "saidaminsaidaxmadov/chocolate-sales",
            "Chocolate Sales (2).csv"
        )
        df.to_csv(dataset_path, index=False)
        print("df: ", df.shape)
    else:
        print("Load predownloaded dataset")


def clear_data(path2data: Path = get_dataset_path()) -> pd.DataFrame:
    def ordinal_encode(orig_df: pd.DataFrame, cols: list[str]) -> pd.DataFrame:
```

```
        df = orig_df.copy()
        df = df.reset_index(drop=True)
        ordinal = OrdinalEncoder(handle_unknown='use_encoded_value',
        ↪  unknown_value=1337)
        ordinal.fit(df[cat_columns])
        ordinal_encoded = ordinal.transform(df[cat_columns])
        df_ordinal = pd.DataFrame(ordinal_encoded, columns=cat_columns)
        df[cat_columns] = df_ordinal[cat_columns]
        return df


    df = pd.read_csv(
        str(path2data), date_format="%d/%m/%Y", parse_dates=["Date"]
    )

    cat_columns = ["Sales Person", "Country", "Product"]
    num_columns = ["Amount", "Boxes Shipped", "Date"]

    # Анализ и очистка данных
    df["Amount"] = df["Amount"].str.replace("$", "")
    df["Amount"] = df["Amount"].str.replace(",", "")
    df["Amount"] = df["Amount"].astype(np.float32)
    df["Date"] = df["Date"].astype('int64') // 10**9

    # Категоризация данных
    df = ordinal_encode(df, cat_columns)
    return df


def featurize(df: pd.DataFrame, config: dict) -> None:
    logger.info("Create features")
    df['amount_per_box'] = df.Amount / df['Boxes Shipped']

    features_path = config['featurize']['features_path']
    df.to_csv(features_path, index=False)


if __name__ == '__main__':
    cfg = load_config(get_dvc_config_path())
    download_data()
    df_prep = clear_data(cfg['data_load']['dataset_csv'])
    df_new_features = featurize(df_prep, cfg)
```

1.2 Обучение модели

Скрипт обучения модели:

```
from typing import Any

import mlflow
from mlflow.models import infer_signature
import numpy as np
import pandas as pd
import pickle
```

```
from sklearn.preprocessing import StandardScaler, PowerTransformer
from sklearn.linear_model import SGDRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.pipeline import Pipeline


def eval_metrics(actual, pred):
    rmse = np.sqrt(mean_squared_error(actual, pred))
    mae = mean_absolute_error(actual, pred)
    r2 = r2_score(actual, pred)
    return rmse, mae, r2


def scale_frame(frame: pd.DataFrame, target: str ="Amount"):
    df = frame.copy()
    X, y = df.drop(columns = [target]), df[target]

    with open('x_train', "w") as f:
        f.write(str(X.to_numpy()))

    scaler = StandardScaler()
    power_trans = PowerTransformer()
    X_scale = scaler.fit_transform(X.values)
    Y_scale = power_trans.fit_transform(y.values.reshape(-1,1))
    return X_scale, Y_scale, power_trans


def train(cfg: dict[str, Any], target: str = 'Amount'):
    df_train = pd.read_csv(cfg['data_split']['trainset_path'])
    df_test = pd.read_csv(cfg['data_split']['testset_path'])
    print(df_train.shape)

    X_train, y_train = df_train.drop(columns=[target]).values,
    ↪  df_train[target].values
    X_val, y_val = df_test.drop(columns=[target]).values, df_test[target].values
    power_trans = PowerTransformer()
    y_train = power_trans.fit_transform(y_train.reshape(-1, 1))
    y_val = power_trans.transform(y_val.reshape(-1, 1))

    mlflow.set_experiment("Linear model Chocolate sales")
    with mlflow.start_run():
        lr_pipe = Pipeline([
            ('scaler', StandardScaler()),
            ('model', SGDRegressor(random_state=42))
        ])
        params = {
            'model__alpha': cfg['train']['alpha'],
            'model__fit_intercept': [False, True],
        }

        clf = GridSearchCV(lr_pipe, params, cv = cfg['train']['cv'], n_jobs=4)
        clf.fit(X_train, y_train.reshape(-1))
        best = clf.best_estimator_
```

```
        best_lr = best['model']

        y_pred = best.predict(X_val)
        y_price_pred = power_trans.inverse_transform(y_pred.reshape(-1, 1))
        print(f'y_price_pred: {y_price_pred[:5]}')
        print(f'y_val: {y_val[:5]}')

        (rmse, mae, r2) =
        ↪  eval_metrics(power_trans.inverse_transform(y_val.reshape(-1, 1)),
        ↪  y_price_pred.reshape(-1, 1))
        mlflow.log_metric("rmse", rmse)
        mlflow.log_metric("r2", r2)
        mlflow.log_metric("mae", mae)
        print(
            [f"R2: {r2}", f"RMSE: {rmse}", f"MAE: {mae}"], sep='\n'
        )

        preds = best.predict(X_train)
        signature = infer_signature(X_train, preds)
        mlflow.sklearn.log_model(best, "model", signature=signature)

        with open(cfg['train']['model_path'], "wb") as file:
            pickle.dump(best, file)

        with open(cfg['train']['power_path'], "wb") as file:
            pickle.dump(power_trans, file)
```

1.3 Оценка качества работы модели

В ходе лабораторной работы акцент в действиях был на получение опыта работы с Data Version Control, а не достижение хороших значений метрик.

Метрики после выполнения пайплайна выводятся в консоль: